



10 Praktycznych Przypadków Użycia Terraform i OpenTofu

10 Praktycznych Przypadków Użycia Terraform i OpenTofu

Wstęp

Terraform i OpenTofu przyda się wielu osobom, które na co dzień w jakiś sposób są odpowiedzialne za zarządzanie infrastrukturą. Oto kilka przykładów takich zastosowań na konkretnych stanowiskach:

- **Software Architect / Software Engineer** - zarządzanie różnymi elementami infrastruktury, np. systemami kolejki wiadomości czy klastrami Kubernetes, gdy np. nie ma dedykowanego DevOps Engineera, czy usługami do monitorowania, gdy nie mamy osobnych osób czy zespołu do monitoringu
- **Cloud Security Engineer** - zarządzanie dostępami do usług chmurowych, np. IAM czy Active Directory
- **Database Engineer** - zarządzanie takimi usługami jak m.in. Google CloudSQL, Amazon RDS, Azure Cosmos DB
- **DevOps Engineer** - zarządzanie np. systemami kolejki wiadomości takimi jak Google Cloud Pub/Sub, Amazon SQS czy Azure Queue Storage czy klastrami Kubernetes
- **Network Engineer** - zarządzanie m.in. sieciami, cloud routerami czy load balancerami
- **Monitoring Engineer** - zarządzanie narzędziami do monitorowania m.in. GCP Stackdriver, Amazon CloudWatch czy Azure Monitor.

Jaką przewagę daje nam Terraform?

- Szybkość wdrażania zmian na wielu usługach na raz
- Szybkość w rekonstrukcji infrastruktury uszkodzonej np. w ataku ransomware czyli „disaster recovery”
- Łatwiejsza kontrola i monitorowanie zmianami w infrastrukturze
- Rozwinięte community pozwala na szybkie uporanie się z problemami technicznym
- Przyspieszenie pracy zespołu
- Łatwiejsze uniknięcie problemów w fazie implementacji

Czym jest OpenTOFU?

OpenTofu jest forkiem Terraforma od wersji 1.5.x, który zyskuje na sławie. Stworzono go, aby zapewnić zastępcze rozwiązanie i alternatywę dla Terraforma, dlatego w obecnych czasach warto znać oba rozwiązania.

Projekt OpenTofu jest zarządzany przez Linux Foundation i jest odpowiedzią na zmiany ze strony firmy Hashicorp, która podjęła decyzję o przeniesieniu Terraforma z Mozilla Public License (MPL) na Business Source License (BSL)

w celach komercyjnych. Projekt OpenTofu działa obecnie na Mozilla Public License.

W tej publikacji zobaczycie przykłady konfiguracji w OpenTofu, abyście zobaczyli, że nie różni się w samej konstrukcji kodu od Terraforma. Główne różnice jakie możemy zaobserwować to komendy z jakich korzysta się pracując z OpenTofu, lecz możliwe, że na przestrzeni najbliższych miesięcy to się zmieni. Aktualizacje odnośnie zmian w OpenTofu będą pojawiać się w projekcie Terraform Maestro.



Link do projektu: <https://github.com/opentofu/opentofu/tree/main>

Zanim zaczniemy

1. Zarówno Terraform jak i OpenTofu korzystają z **Hashicorp Configuration Language (HCL)** czyli języka stworzonego przez Hashicorp. Z tego względu struktura konfiguracji w obu przypadkach praktycznie się nie różni na obecnym etapie. Projekt OpenTofu jest świeży i cały czas się rozwija, dlatego w projekcie Terraform Maestro będą pojawiały się update'y kodu.
2. W trakcie tworzenia usług, będziemy korzystać z nazwy **violet** inspirowaną kolorem logo Terraforma.
3. Wszystkie przykłady kodu, niezależnie czy to Terraforma czy OpenTofu, powinny zostać zapisane w pliku **main.tf**.

CASE 1: Platform Engineer / Cloud Engineer

Jeśli na co dzień zajmujesz się zarządzaniem sieciami chmurowymi, Terraform może bardzo usprawnić Twoją pracę. Dzięki możliwościom tego narzędzia będziesz w stanie z łatwością zarządzać dużą liczbą sieci, load balancerów czy cloud routerów, a także tworzyć reguły firewallowe. Omówmy przykłady zastosowań Terraforma, które mogą pomóc Ci w codziennej pracy. Pokażę też, jak jeden z tych przykładów można opisać w OpenTofu.

1. Tworzenie sieci wirtualnej (VPC) w GCP

Stworzymy nową sieć wirtualną, którą w świecie usług chmurowych nazywamy VPC wraz z podsiecią. Jest to jeden z pierwszych kroków jakie wykonujemy po utworzeniu nowego projektu w chmurze.

```
provider "google" {
  project = "id"
  region  = "eu-west1"
}

resource "google_compute_network" "violet_network" {
  name                  = "violet-network"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "violet_subnet" {
  name      = "violet-subnet"
  region    = "eu-west1"
```

```

network      = google_compute_network.violet_network.self_link
ip_cidr_range = "10.0.1.0/24"
}

```

W pierwszej kolejności definiujemy providera, czyli w tym wypadku google, gdyż chcemy postawić sieć w GCP. W tej sekcji definiujemy jedynie 2 pozycje, czyli numeru ID projektu oraz region, w którym stawiamy sieć. Kolejny krok to wykorzystanie zasobu **google_compute_network**. Dzięki niemu stworzymy nową sieć wirtualną (VPC) o nazwie *violet-network*. Ostatnim krokiem jest wykorzystanie zasobu **google_compute_subnetwork** do stworzenia podsieci.

Dla osób niezaznajomionych z tworzeniem sieci w chmurze oto kilka informacji, o których warto pamiętać na tym etapie:

- każda sieć VPC jest podzielona na podsieci,
- każda podsieć znajduje się w jednym regionie,
- można mieć więcej niż jedną podsieć w regionie dla danej sieci VPC,
- w podsieciach stawiamy np. instancje czy kontenery.

Nasza podsieć nazywa się *violet-subnet*, będzie znajdowała się w Europie, a konkretnie w regionie *eu-west1*. Pula adresów będzie mieściła się w zakresie *10.0.1.0/24*. Oznacza to, że w stworzonej podsieci będzie dostępnych łącznie 256 adresów.

O co chodzi z **network = google_compute_network.violet_network.self_link**?

Google_compute_network.violet_network.self_link odnosi się do właściwości **self_link** zasobu *violet_network*, czyli sieci o nazwie *violet-network*. Tym sposobem uzyskujemy dostęp do identyfikatora URL tej sieci, aby następnie przekazać go do parametru **network** zasobu **violet_subnet**, czyli podsieci o nazwie *violet-subnet*. Dzięki temu podsieć zostanie powiązana z siecią.



Linki do zasobów:

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_network

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_subnetwork

2. Tworzenie sieci wirtualnej (VPC) w AWS

Stworzyliśmy sieć w GCP, a teraz dla porównania zobaczmy jak to się robi w AWS.

```

provider "aws" {
  region = "eu-west-1"
}

resource "aws_vpc" "violet_vpc" {

```

```

    cidr_block = "10.0.0.0/16"
  }

  resource "aws_subnet" "violet_subnet" {
    vpc_id      = aws_vpc.violet_vpc.id
    cidr_block  = "10.0.1.0/24"
    availability_zone = "eu-west-1a"
  }

```

W przypadku chmury AWS sytuacja jest bardzo podobna, czyli w sekcji provider definiujemy AWS i region. Następnie tworzymy sieć VPC, dzięki zasobowi **aws_vpc**.

Zakres adresów IP w sieci *violet_vpc* definiujemy jako **cidr_block** i w naszym przypadku jest to **10.0.0.0/16**. Oznacza to, że utworzone przez nas VPC będzie zawierać maksymalnie 65534 adresy IP.

Ostatnim zasobem jaki wykorzystujemy jest **aws_subnet** i posłuży nam do stworzenia podsieci.

Argumentowi **vpc_id** przypisujemy wartość **aws_vpc.violet_vpc.id**

W ten sposób definiujemy, że nowa podsieć należy do VPC o nazwie *violet_vpc* czyli do tej stworzonej wcześniej przy pomocy zasobu **aws_vpc**.

Cidr_block = "10.0.1.0/24" oznacza, że podsieć może zawierać do 251 adresów IP.

Availability_zone = "eu-west-1a" oznacza, że podsieć będzie w regionie EU (Irlandia).



Linki do zasobów: <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpc>
<https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/subnet>

3. Tworzenie Security Group w AWS

Stwórzmy Security Group w AWS, która może nam posłużyć jako mechanizm kontroli dostępu na poziomie instancji EC2 oraz innych usługach, takich jak np. Elastic Load Balancer. Jest to rodzaj firewalla, która kontroluje ruch sieciowy wychodzący i wchodzący do danego zasobu (w naszym przypadku instancji EC2).

```

resource "aws_security_group" "violet_security_group" {
  name          = "violet-security-group"
  description   = "Violet security group"
  vpc_id        = aws_vpc.violet_vpc.id

  ingress {
    from_port    = 80
    to_port      = 80
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

```

```

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

```

Tworzymy security grupę o nazwie **violet-security-group** z wykorzystaniem zasobu **aws_security_group**. Możemy dodać opis grupy, co warto robić w przypadku zarządzania dużą ilością grup. Security grupę tworzymy w **violet_vpc**, dzięki **aws_vpc.violet_vpc.id**. Jest to VPC, które stworzyliśmy w poprzednim przykładzie.

Jako przykład zdefiniowaliśmy regułę ingress zezwalającą na połączenia przychodzące do portu HTTP (80) z dowolnego adresu czyli "0.0.0.0/0", a także regułę egress, która umożliwi ruch sieciowy wychodzący na dowolny port i protokół do dowolnego miejsca.

Co oznacza wartość **protocol = "-1"** oznacza, że reguła egress dotyczy wszystkich protokołów.

W przypadku tej reguły, wartość **-1** dla pola **protocol** oznacza, że nie jest ona ograniczona do żadnego konkretnego protokołu, ale zezwala na ruch dla wszystkich typów protokołów sieciowych, takich jak TCP, UDP czy ICMP. Przepuszcza ona ruch sieciowy niezależnie od tego czy jest to pingowanie (ICMP), wchodzenie na stronę internetową (HTTP/HTTPS) czy inny przypadek.



Link do zasobu:

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/security_group

4. Tworzenie AWS Route 53

Usługa Route 53 oferuje nam wiele możliwości takich jak rejestracja domen, zarządzanie rekordami DNS, różne możliwości routingu, a także monitorowanie stanu zasobów DNS.

Stworzymy strefę DNS i nowy rekord z pomocą Terraforma.

```

provider "aws" {
  region = "us-east-1"
}

resource "aws_route53_zone" "violet-zone" {
  name = "violet.com"
}

resource "aws_route53_record" "violet-record" {
  zone_id = aws_route53_zone.violet-zone.zone_id
  name    = "www.violet.com"
  type    = "CNAME"
}

```

```

ttl      = 300
records = ["not-so-violet.com"]
}

```

Dzięki zasobowi **aws_route53_zone** tworzymy strefę DNS o nazwie *violet.com*. Kolejnym krokiem jest utworzenie nowego rekordu CNAME w tej strefie o nazwie *violet-record* i zmapowanie adresu *www.violet.com* na adres *not-so-violet.com*.



Linki do zasobów:

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/route53_zone

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/route53_record

5. Tworzenie bucketa AWS S3

AWS S3 jest jedną z najbardziej popularnych usług. Nazwa S3 pochodzi od Simple Storage Service i pozwala nam ona na przechowywanie różnych zasobów takich jak np. obrazy, multimedia czy pliki. Zastosowań może być wiele od statycznych stron typu landing page po backupy. Na przykładzie S3 zobaczycie również różnice pomiędzy konfiguracją w Terraformie, a OpenTofu.

Terraform

```

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "violet_bucket" {
  bucket = "violet-bucket"
  acl    = "private"

  tags = {
    name          = "violet-bucket"
    environment = "Dev"
  }
}

```

Standardowo definiujemy providera, następnie tworzymy S3 bucket o nazwie *violet-bucket* za pomocą zasobu **aws_s3_bucket**. Dodaliśmy też tagi oraz ustawiliśmy dostęp do bucketa (acl) na prywatny, aby nasz bucket nie był dostępny publicznie.



W obecnych czasach, u większości providerów buckety domyślnie są prywatne, jednak pamiętaj by ustawić ten parametr w swojej konfiguracji. Wystawione publicznie buckety są jednym z najczęściej spotykanych powodów wycieków danych ze środowisk chmurowych.



Link do zasobu:

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/s3_bucket

OpenTofu

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "violet_bucket" {
  bucket = "violet-bucket"
  acl    = "private"

  tags = {
    name          = "violet-bucket"
    environment = "Dev"
  }
}
```

Jak łatwo zauważycie, struktura OpenTofu nie różni się od Terraforma. Natomiast to jak implementujemy te zmiany w praktyce korzystając z takich komend jak **tofu plan** czy **tofu apply** są głównymi różnicami.

CASE 2: Database Engineer

Jeśli na co dzień zajmujesz się zarządzaniem bazami danych, pokaże Ci jak w prosty sposób można stworzyć bazodanowe usługi przy pomocy Terraforma.

6. Tworzenie Microsoft SQL Azure Database Server

Stwórzmy serwer Microsoft SQL Server na platformie Azure, jako, że jest to często kluczowy element wielu aplikacji, umożliwiający przechowywanie i zarządzanie danymi w relacyjnych bazach danych.

Usługa **Microsoft SQL Azure Database Server** nazywana też Azure SQL Database to w pełni zarządzana relacyjna baza danych jako usługa DBaaS na platformie Azure. Dostarcza nam ona wszystkich funkcji Microsoft SQL Server, ale bez potrzeby zarządzania infrastrukturą, np. dbania o skalowanie.

W kodzie zdefiniowane są niezbędne zasoby, takie jak dostawca Azure, serwer SQL oraz przypisanie do odpowiedniej grupy zasobów.

```

provider "azurerm" {
}

resource "azurerm_resource_group" "violet-database" {
  name      = "violet-database-rg"
  location  = "West Europe"
}

resource "azurerm_mssql_server" "violet-database" {
  name                               = "violet-mssqlserver"
  resource_group_name                = azurerm_resource_group.violet-database.name
  location                           = azurerm_resource_group.violet-database.location
  version                            = "12.0"
  administrator_login                 = "<login>"
  administrator_login_password       = "<password>"
  minimum_tls_version                 = "1.2"

  tags = {
    environment = "production"
  }
}

```

Definiujemy providera jako **azurerm**.

Następnie korzystamy z **azurerm_resource_group**, aby zdefiniować nazwę i lokalizację, która następnie jest wykorzystywana w zasobie **azurerm_mssql_server** jako *resource_group_name* i *location*.

Baza, którą tworzymy z wykorzystaniem zasobu **azurerm_mssql_server**, będzie miała nazwę *violet-database*, a maszyna będzie nosiła nazwę *violet-mssqlserver*.

Resource_group_name określa nazwę grupy zasobów do jakiej będzie przypisany nasz nowy serwer SQL.

Pobieramy ją z poprzednio stworzonego zasobu **azurerm_resource_group**, co oznacza, że będzie to nazwa *violet-database-rg*.

Dzięki **location** określamy region, w którym maszyna będzie stała. Lokalizacja serwera zostanie ustawiona na tę samą lokalizację co grupa zasobów *violet-database* czyli *West Europe*.

Nasz SQL Server będzie w wersji 12.0, a minimalna dopuszczalna wersja TLS to 1.2.

Definiujemy także **login** i **hasło** administratora.



Oczywiście trzymanie ich w wersji plain text w tym miejscu nie jest zgodne z praktykami bezpieczeństwa. Jest to tylko przykład prostego zastosowania Terraforma, jednak w ramach projektu Terraform Maestro poruszymy także temat bezpiecznego przechowywania danych wrażliwych.



Link do zasobu:

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/mssql_server

7. Tworzenie Google Cloud SQL

Tym razem stwórzmy instancję bazy danych MySQL w GCP.

Cloud SQL to zarządzana usługa bazodanowa oferowana przez Google Cloud Platform, która umożliwia uruchamianie relacyjnych baz danych w chmurze. Obejmuje ona popularne systemy zarządzania bazami danych takie jak MySQL, PostgreSQL czy SQL Server.

```
provider "google" {
  project      = "id"
  region      = "us-central1"
}
resource "google_sql_database_instance" "violet_sql" {
  name          = "violet-sql-instance"
  database_version = "MYSQL_8_4"
  region        = "us-central1"

  settings {
    tier = "db-f1-micro"
  }
}
```

Korzystając z zasobu **google_sql_database_instance** możemy stworzyć instancję Cloud SQL w GCP. Zgodnie z opisem zasobu, będzie to instancja o nazwie *violet-sql-instance*, na której zostanie zainstalowana baza danych MySQL w wersji 8.4 i stanie ona w regionie us-central1 co oznacza północną część Ameryki.

Typ maszyny do naszych testów to **db-f1-micro**. Maszyna db-f1-micro daje nam 0.6 GB pamięci RAM i około 3 GB maksymalnej pojemności pamięci dysku.



Więcej na ten temat rodzajów maszyn dla usługi Cloud SQL: <https://cloud.google.com/sql/pricing>



Link do zasobu:

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/sql_database_instance

CASE 3: DevOps Engineer

Pracując jako DevOps Inżynier często ma się do czynienia z różnego rodzaju usługami chmurowymi takimi jak choćby klastry Kubernetes stawiane jako EKS, AKS czy GKE. Z pomocą Terraforma możemy w stosunkowo łatwy sposób zarządzać tworzeniem klastów Kubernetes czy usługami do kolejkowania.

8. Tworzenie klastra Kubernetes (Google Kubernetes Engine) w GCP

```
resource "google_service_account" "default" {
  account_id    = "id"
  display_name = "Service Account for Kubernetes"
}

resource "google_container_cluster" "primary" {
  name                = "violet-cluster"
  location            = "eu-north1-b"
  initial_node_count = 2
  node_config {
    service_account = google_service_account.default.email
    oauth_scopes = [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
    labels = {
      env = "dev"
    }
  }

  autoscaling {
    min_node_count = 2
    max_node_count = 10
  }

  machine_type = "e2-medium"
  disk_size_gb = 50
  disk_type    = "pd-standard"

  network      = "violet-network"
  subnetwork   = "violet-subnet"

  enable_private_nodes = true
  master_authorized_networks_config {
    cidr_blocks = ["10.0.0.0/24"]
  }
}

timeouts {
  create = "30m"
  update = "40m"
}
```

Na początek potrzebne nam będzie nowe konto usługi w GCP czyli **Google Service Account**. Możemy je stworzyć dzięki zasobowi **google_service_account**.

Następnym krokiem jest stworzenie klastra Kubernetes w usłudze GKE czyli odpowiednika dla amazonowego EKS czy azurowego AKS. Zgodnie z naszą konfiguracją klastr zostanie stworzony w Europie Północnej, na co wskazuje lokalizacja regionu *eu-north1-b*. Klastr wystartuje z 2 node'ami, które będą działać na service account utworzonym wcześniej.

W parametrze **oauth_scopes** definiujemy uprawnienia OAuth, które są używane do kontrolowania dostępu do zasobów usługi przez aplikacje lub użytkowników.



Więcej na temat OAuth: <https://developers.google.com/identity/protocols/oauth2?hl=pl>

Klastry będą oznaczone labelami jako środowiska *dev*.

W sekcji autoscaling definiujemy minimalną i maksymalną ilość node'ów w celu kontroli automatycznego skalowania klastra w zależności od obciążenia. Dzięki temu ustawieniu, GCP automatycznie doda nowe node'y, gdy wzrośnie obciążenie klastra. Gdy obciążenie spadnie, system usunie zbędne node'y, co chroni nas przed niepotrzebnymi kosztami za zasoby, które nie są wykorzystywane w danym momencie.

W dalszej części konfiguracji definiujemy rodzaj maszyny (**machine_type**), wielkość dysku (**disk_size_gb**), rodzaj dysku (**disk_type**).



Więcej o:

- rodzajach maszyn w GCP przeczytasz tu: <https://cloud.google.com/compute/docs/general-purpose-machines>
- rodzajach dysków w GCP przeczytasz tu: <https://cloud.google.com/compute/docs/disks>

Następnie stawiamy nasz klastr w sieci (**network**) o nazwie *violet-subnet* i podsieci (**subnetwork**) o nazwie *violet-subnet*. Odnosimy się tutaj do sieci, którą postawiliśmy na początku książki w **CASE 1: Platform Engineer / Cloud Engineer**.

Dalej włączamy opcję **enable_private_nodes**, dzięki czemu node'y będą dostępne tylko z wewnętrznej sieci VPC,

Jako

master_authorized_networks_config określamy, zakres CIDR adresów jakie mają uprawnienie do bezpośredniego dostępu do klastra Kubernetes co tak naprawdę oznacza, że tylko komputery wskazane w tym parametrze będą miały uprawnienie do łączenia się z klastrami Kubernetes.

W sekcji **timeouts** ustawiamy limity czasowe do 30 minut przy tworzeniu klastra i 40 minut przy update'owaniu.

Po wdrożeniu tej konfiguracji przez Terraform, będziemy mieli funkcjonujący klastr GKE, wraz z nowym service account, który może być używany do zarządzania dostępem do zasobów w chmurze Google.



Linki do zasobów:

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/google_service_account

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/container_cluster

9. Tworzenie AWS Elastic Beanstalk

Jak szybko wdrożyć aplikację w AWS?

Dzięki usłudze Elastic Beanstalk, która wspiera aplikacje oraz usługi napisane w różnych językach, takich jak Java, .NET, PHP, Node.js, Python, Ruby, Go oraz Docker, na takich serwerach jak Apache czy Nginx. Jest to kolejny przykład, na którym zobaczycie również różnice pomiędzy konfiguracją w Terraformie, a OpenTofu.

Terraform

```
provider "aws" {
  region = "eu-east-2"
}

resource "aws_elastic_beanstalk_application" "violet_app" {
  name = "violet-app"
}

resource "aws_elastic_beanstalk_environment" "violet_env" {
  name                = "violet-env"
  application          = aws_elastic_beanstalk_application.violet_app.name
  solution_stack_name = "64bit Amazon Linux 2 v5.4.5 running Node.js 18"
}
```

W pierwszej kolejności stosujemy zasób **aws_elastic_beanstalk_application**. Stworzy on aplikację Elastic Beanstalk o nazwie *violet-app*.

Następnie korzystamy z zasobu **aws_elastic_beanstalk_environment**, by stworzyć środowisko czyli miejsce, gdzie uruchamiana będzie aplikacja *violet-app*. Jako *application* definiujemy aplikację stworzoną w zasobie **aws_elastic_beanstalk_application**.

Solution_stack_name definiuje system operacyjny i środowisko do uruchomienia aplikacji. W tym przypadku wykorzystamy Amazon Linux i Node.js.



Linki do zasobów:

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/elastic_beanstalk_application

https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/elastic_beanstalk_environment

OpenTofu

```

provider "aws" {
  region = "eu-east-2"
}

resource "aws_elastic_beanstalk_application" "violet_app" {
  name = "violet-app"
}

resource "aws_elastic_beanstalk_environment" "violet_env" {
  name                = "violet-env"
  application         = aws_elastic_beanstalk_application.violet_app.name
  solution_stack_name = "64bit Amazon Linux 2 v5.4.5 running Node.js 18"
}

```

Jak łatwo zauważycie, struktura OpenTofu nie różni się od Terraforma. Natomiast to jak implementujemy te zmiany w praktyce korzystając z takich komend jak **tofu plan** czy **tofu apply** są głównymi różnicami.

10. Tworzenie Azure Service Bus

Stwórzmy infrastrukturę Azure Service Bus składającą się z grupy zasobów, namespace'a, topica oraz subskrypcji. Taka konfiguracja pozwoli nam na realizację scenariuszy komunikacyjnych typu publish/subscribe, gdzie różne usługi mogą publikować wiadomości do topica, a inne usługi mogą subskrybować i odbierać te wiadomości.

Jest to usługa stosowana przez zespoły developerów i devopsów i spotyka się ją często w architekturze mikroservisów oraz innych systemach rozproszonych.

```

provider "azurerm" {
}

resource "azurerm_resource_group" "violet_rg" {
  name      = "violet-resources"
  location = "West EU"
}

resource "azurerm_servicebus_namespace" "violet_namespace" {
  name                = "violet-servicebus-namespace"
  location            = azurerm_resource_group.violet_rg.location
  resource_group_name = azurerm_resource_group.violet_rg.name
  sku                 = "Basic"
}

resource "azurerm_servicebus_topic" "violet_topic" {
  name                = "violet-topic"
  namespace_name      = azurerm_servicebus_namespace.violet_namespace.name
  resource_group_name = azurerm_resource_group.violet_rg.name
}

```

```
resource "azurerm_servicebus_subscription" "violet_subscription" {
  name                = "violet-subscription"
  topic_name          = azurerm_servicebus_topic.violet_topic.name
  namespace_name      = azurerm_servicebus_namespace.violet_namespace.name
  resource_group_name = azurerm_resource_group.violet_rg.name
}
```

Na początek poza zdefiniowaniem providera, musimy także stworzyć grupę zasobów dzięki zasobowi **azurerm_resource_group**. Ułatwi nam to zarządzanie, monitorowanie i rozliczanie zasobów jako jednostki.

Następnie wykorzystując zasób **azurerm_servicebus_namespace** tworzymy namespace dedykowany usłudze Service Bus. Namespace będzie miał nazwę *violet-servicebus-namespace* i będzie działał jako kontener dla topic i subskrypcji oraz umożliwi nam zarządzanie oraz zabezpieczanie zasobów.

Czym jest SKU i co oznacza sku = "Basic"?

SKU to skrót od "Stock Keeping Unit" i jest to rodzaj planu/poziomu usługi, który jest wykorzystywany w danej usłudze, a w tym konkretnym przypadku w usłudze **azurerm_servicebus_namespace**. **Sku = "Basic"** oznacza, że plan usługi dla tego namespace jest w wersji podstawowej, co oznacza ograniczone funkcje i niższe koszty.

Kolejny krok to stworzenie topica Service Bus o nazwie *violet-topic* w namespace *violet-servicebus-namespace* oraz w grupie zasobów *violet-resources*.

Ostatni krok to stworzenie subskrypcji Service Bus o nazwie *violet-subscription* dla tematu *violet-topic* w namespace *violet-servicebus-namespace* w grupie zasobów *violet-resources*.



Linki do zasobów:

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/resource_group

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/servicebus_namespace.html

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/servicebus_topic.html

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/servicebus_subscription

Podsumowanie

Terraform i OpenTofu to narzędzia, dzięki którym możesz zautomatyzować tworzenie infrastruktury. Przykłady jakie zaprezentowałam są proste, ponieważ celem tej publikacji było zaprezentowanie możliwości zastosowania tych narzędzi w konkretnych use case'ach.

Oba narzędzia mają jednak dużo więcej do zaoferowania. W projekcie Terraform Maestro rozwiiniemy niektóre z tych przykładów, m.in.:

- poznamy dodatkowe funkcjonalności obu narzędzi, aby np. nie musieć hardcodować tych samych wartości w kilku miejscach, lecz definiować je dynamicznie,
- zadamy o poprawne przechowywanie danych wrażliwych takich jak loginy i hasła,
- nauczymy się pracować z różnymi wersjami kodu, tak aby można było cofnąć się do poprzedniej, jeśli np. wdrożenie nowej wersji coś popsuło,

- dodamy zostaną testy sprawdzające poprawność konfiguracji,
- nauczymy się debugować problemy związane z konfiguracją,
- poznamy dużo tricków, które pozwolą wam na zarządzanie ogromną infrastrukturą w prosty sposób.

Jako dodatkowy plus projektu, zagłębimy się nie tylko w temat Terraforma i OpenTofu, ale także świat rozwiązań chmurowych takich jak GCP, AWS czy MS Azure, a przy okazji poznamy również dobre praktyki odnośnie do architektury infrastruktury chmurowej.

Autorka publikacji



Katarzyna Brzozowska - DevSecOps Engineer

Nadzór i redakcja



Damian Naprawa - Docker Captain, Cloud & Kubernetes Architekt, Założyciel wkontenerach.pl

Chcesz podać e-book dalej? Prześlij znajomemu tę stronę: www.TFMaestro.pl